

Your software factory already has a data model

The agent can build the model. I still want a human deciding what it means.

Aug 21, 2026

My software factory had closed 12,225 work items, and my agent coordinating the flow of work had acknowledged 116 of their outcomes.

These are numbers from the same snapshot in time, and both are plausible values that could be returned for a question like, “how is our software factory doing?”

The “work items closed” value is a producer claim: a worker, controller, or cleanup path had moved a record to `closed`. The second goes a step further and describes an outcome that traveled through a coordinator-agent’s ownership and reached an `acknowledged` state. They don’t describe the same fact.

I decided to test how an agent would reconcile semantic ambiguity like this. I launched an `Amp` agent in a new `Orb` and had it turn my factory’s operational history into a dbt project, model it in Omni, then use Omni in the browser to build and inspect a control-room dashboard.

Agents are totally capable of writing correct SQL, but I wanted to see what happened when the agent also had to decide which of those two numbers meant **successful work**.

I don’t really want my agent deciding that for me in unpredictable ways.

I am happy to let the agent trace the implementation, write the transformations and tests, propose the semantic model, document its reasoning, build the dashboard, and use the application to check what a person will actually see. But when the evidence supports more than one defensible interpretation, I want the ambiguity surfaced for review rather than collapsed into a metric.

The factory had already modeled itself

My source system runs using `Gas City`, a software-factory SDK I help maintain. For this experiment, I took an August 20 snapshot containing 17,110 work items, 13,035 scheduled-order runs, and 14,544 agent runs.

None of the data tables from my factory’s operational history were designed for business intelligence. They exist because the factory needs them to coordinate work: claims tell workers what they own, events preserve transitions, outcome envelopes track whether produced work made it back to the coordinating agent, and usage journals record model calls.

There was plenty of structure there to build an analytical model, and plenty of ways to get the meaning wrong.

For example, every scheduled order in the snapshot has `status = 'closed'`. That sounds promising until you notice that 2,837 of those same records have a dispatcher result of `failed`, and another eight are incomplete. Here, `closed` tells me that the record reached the end of its lifecycle. It does not tell me that the work succeeded.

Cost has a different problem. I can account for \$373.20 of fully priced model usage and another \$68.60 as a lower bound from runs where only some of the usage is priced. But I can’t reliably join every run back to a specific work item. An agent could still produce a “cost per successful task” metric, but doing so would require inventing a relationship the source system doesn’t actually contain.

Time is messy too. Some clients wrote UTC timestamps while others wrote Eastern wall-clock time. Once those values are sitting next to each other in a column, there isn’t enough information in the timestamp alone to recover which convention produced each row.

So the hard part of agent-authored analytics starts before the agent ever opens Omni. The source system already contains a lot of structure, but structure is not the same thing as meaning. A field

name can suggest an interpretation; the implementation and surrounding system behavior are what tell me whether I should trust it.

I gave the agent the whole job

I used `Amp Orbs` because I wanted to see how much of this workflow a cloud agent could actually own. I launched an Amp agent in a fresh Orb with a checkout containing the factory snapshot, dbt project, relevant source code, and a lightweight style guide for the final dashboard: KPI hierarchy, layout, labels, and visual treatment.

Its job was not just to write a few transformations. It had to understand the source system well enough to model it, turn that model into something usable in Omni, and then use Omni itself to build and inspect the resulting dashboard.

That included working through the ambiguities in the underlying data. The agent kept `closed` and coordinator acknowledgement as separate facts rather than treating them as synonyms. It treated partially priced model usage as a lower bound rather than a complete cost number. When the source system did not contain enough evidence to support a definition, it left the question unresolved instead of inventing one.

Then it carried those decisions forward into Omni.

The agent created the analytical fields and descriptions, connected them into a usable `Omni topic`, and encoded guidance about how those fields should be interpreted. It then opened Omni in the browser and built the control-room dashboard a person would actually use.

An agent is capable of much more than generating raw SQL. It can move from source code and operational data, through dbt and Omni’s semantic layer, to the dashboard someone actually uses.

The boundary I care about is approval. The agent can propose and implement a definition, but a human should decide when that definition becomes authoritative.

The browser belongs in the test

Once the model existed, the agent used Omni to turn it into something a person could verify.

It opened an isolated model branch using [Branch Mode](#), loaded the semantic files, built the dashboard tiles, and arranged the control room around the distinctions that mattered most. The first screen puts “claimed closed” next to “coordinator acknowledged,” shows the 12,109-record gap between them, and makes the roughly 105x difference hard to miss. A provenance tile shows when the snapshot was taken, how many rows it contains, and how much of the data carries clock uncertainty.

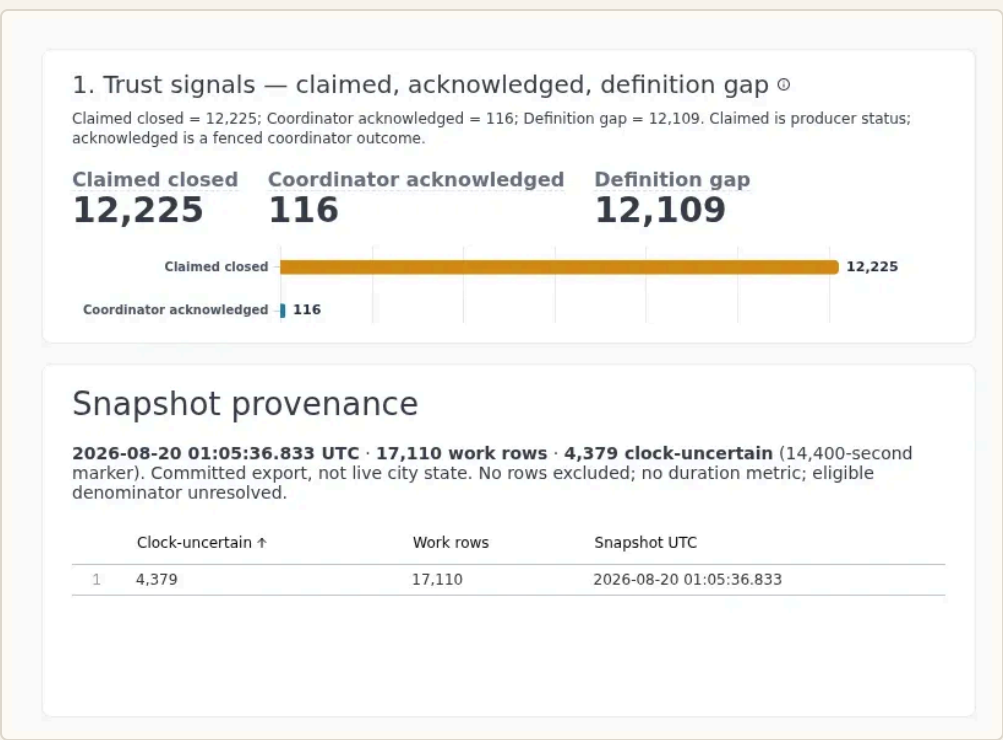
The same definitions also live below the dashboard. Measures have descriptions, joins have explicit grains, and the topic includes guidance about what fields such as `closed` do and do not mean. Every aggregate can drill back to `work_id`.

A correct dbt model can still turn into a misleading dashboard. A measure can be defined correctly and then mislabeled, compared at the wrong grain, or presented without enough context to interpret it. So I wanted the agent to work in Omni itself: change the underlying model, use those fields in the dashboard, inspect the rendered result, and revise what it had built.

I checked the important values three ways: directly against the dbt-built relations in MotherDuck, through Omni’s SQL runner, and from fields exposed through the Omni topic. All three returned the same reference values.

By the end of the run, the agent had built 13 dbt models, passed the model tests, created the semantic layer, and assembled all seven dashboard tiles in Omni.

Figure 01 / Omni control room



The Orb built and styled this in Omni’s browser UI. Large KPIs establish the hierarchy; the zero-based comparison, gap, and provenance keep it honest.

What the reviewer caught

The agent made one decision I would not have approved.

It excluded a work item because its close and start times appeared earlier than its creation time. The model built successfully, the exclusion was documented, and the result was internally consistent.

But the row was not bad data. The roughly four-hour inversion matched the mixed UTC and Eastern timestamp problem in the source system. The right treatment was to keep the row, mark it with `clock_uncertainty_seconds = 14400`, and expose that uncertainty in the dashboard.

I sent the correction back to the agent, and it updated the model and the rendered dashboard.

This is the boundary I was trying to preserve. The agent had enough information to make a reasonable choice, implement it, test it, and explain what it had done. I still needed to decide whether that choice matched what I was willing to claim about the data.

Omni becomes the control substrate

I wanted those definitions to live somewhere shared.

Omni’s [dbt integration](#) can bring dbt models, descriptions, tests, and tags into its semantic layer. From there, the same dimensions, measures, joins, descriptions, and permissions can be used by a person exploring the data, by dashboards, by [Omni’s AI features](#), and by external agents using [Omni MCP](#).

This is where I can separate what the agent is allowed to build from what I have actually approved.

In practice, I can let the agent resolve an ambiguity as far as the evidence takes it: propose the measure, define its SQL and description, expose it through an Omni topic, and build dashboard

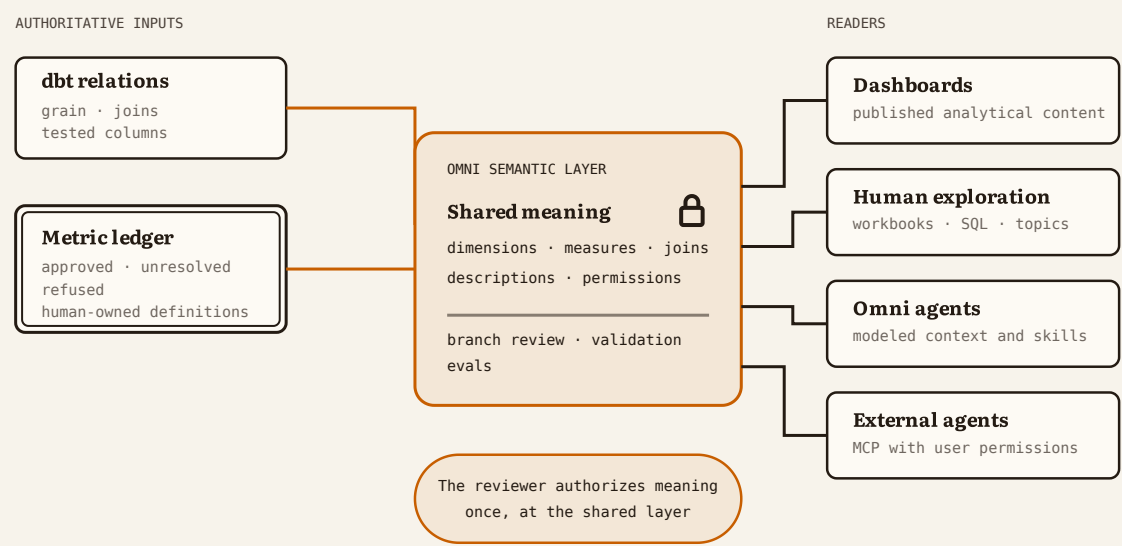
content against it on a branch. I can then review the definition in the context where it will actually be used before deciding whether to merge it into the shared model. If the evidence is not good enough, the agent has still done the implementation work; the definition just does not become authoritative.

Once approved, that definition is no longer an agent-specific interpretation. It becomes part of the same semantic layer used by dashboards, human exploration, Omni’s AI, and external agents over MCP.

The same applies to permissions. An external agent using MCP should not get a different interpretation of the data from the person looking at the dashboard, and it should not get access to more than that person would.

Figure 02 / Control substrate

The semantic layer serves every analytical reader



One reviewed definition reaches dashboards, human exploration, Omni’s agents, and external agents over MCP.

For this experiment I kept the dbt project, Omni model files, evidence, and review artifacts together in one repository. Omni’s [dbt setup guidance](#) recommends separating the dbt and Omni repositories in production, so I would not assume this repo layout is the one I would keep long term.

In this experiment, I left the Omni branch unmerged. The repository also could not prove that the committed export still matched the live factory at review time. Both are important limits on what this experiment actually demonstrated.

The review packet I want

What I want back from an agent is not just a dashboard.

I want the model diff, the source evidence behind consequential definitions, the values it used to validate the result, screenshots of

what it actually built, the questions it could not resolve, and any checks it could not complete.

The workflow is fairly simple: pin the input and its provenance, keep important metric definitions somewhere reviewable, test the transformations, verify that the values agree across dbt and Omni, then have the agent build and inspect the dashboard in the browser.

The agent can do almost all of that work, from tracing the source system through building and revising the dashboard.

What I want to sign off on is the meaning.